U.E 5.1 - Intelligence Artificielle : Machine learning
Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France

BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr
BENIGNO DA SILVA Matheus Henrique
matheus.benigno@ensta.fr

Sommaire

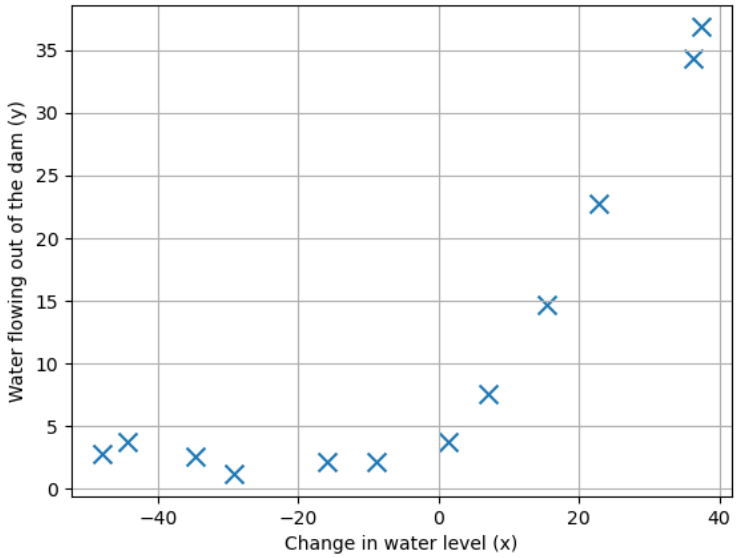
1 Regularized Linear Regression **2**
1.1 Visualizing the dataset 2
1.2 Regularized linear regression cost function 2
1.3 Regularized linear regression gradient 3
1.4 Fitting linear regression 3

2 Bias-variance **4**
2.1 Learning curves 4

3 Polynomial regression **6**

1 Regularized Linear Regression

1.1 Visualizing the dataset



1.2 Regularized linear regression cost function

```python
import numpy as np
def linearRegCostFunction(X, y, theta, Lambda):
    """computes the cost of using theta as the parameter for
       linear regression
    to fit the data points in X and y.
    Returns the cost in J
    """

    # Initialize some useful values

    m,n = X.shape # number of training examples
    theta = theta.reshape((n,1)) # in case where theta is a
       vector (n,)
    J = 0.


    h = X.dot(theta)
    a = (1/(2*m)) * np.sum(np.square(h - y))
    b = (Lambda/(2*m)) * np.sum(np.square(theta[1:]))
    J = a+b

    return J.flatten()
```

- Output :

```
luiz@luiz-Cyborg-15-A13VFK:~/Documents/Machine_Learning/ex4_starterCode$ /usr/bin/python3 /home/luiz/Documents/Machine_Learning/ex4_starterCode/ex4.py
Loading and Visualizing Data ...
Cost at theta = [1  1]: 303.993192
(this value should be about 303.993192)
```

1.3 Regularized linear regression gradient

```python
import numpy as np


def linearRegGradientFunction(X, y, theta, Lambda):
    """computes the gardients of the cost using theta as the
       parameter for linear regression
        to fit the data points in X and y.
        Returns the gradient in grad
     """

    # Initialize some values
    m,n = X.shape # number of training examples
    theta = theta.reshape((n,1)) # in case where theta is a
       vector (n,)
    grad = 0.

    h = X.dot(theta)
    grad = (1/m) * X.T.dot(h - y) + (Lambda/m) * np.r_[[[0]],
       theta[1:]]

    return grad.flatten()
```

- Output :

```
(this value should be about 303.993192)

Gradient at theta = [1  1]:  [-15.303016 598.250744]
(this value should be about [-15.303016 598.250744])
```

1.4 Fitting linear regression

```
... code-block:: python

    # ====================== YOUR CODE HERE
       ======================
    # Instructions: Compute
    #   - theta the final thetas
    #   - the training cost vs iteration number from thetas
    #   - the validation cost vs iteration number from thetas

```

```
    # outputs
    theta = result.x
    cost_train = [linearRegCostFunction(X, y, t, 0) for t in
       thetas]

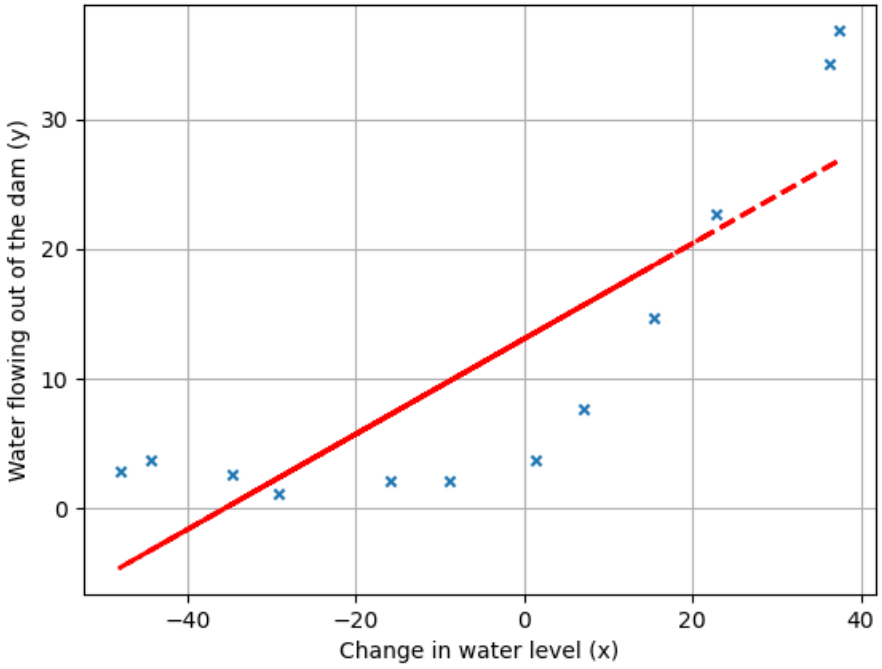
    if Xval is not None and yval is not None:
        cost_val = [linearRegCostFunction(Xval, yval, t, 0)
           for t in thetas]

    # =========================================

    # force to np array
    cost_train = np.array(cost_train)
    cost_val = np.array(cost_val)

    return theta, cost_train, cost_val
```

- Output :



2 Bias-variance

2.1 Learning curves

```
... code-block:: python

def learningCurve(X, y, Lambda, Xval, yval, ax=plt.gca(),
    maxiter=2000):
    """returns the train and validation/test set costs for a
       learning curve. In particular,
    it returns two vectors of the same length - cost_train and
    cost_val.

    In this function, you will compute the train and
       validation costs.
    """

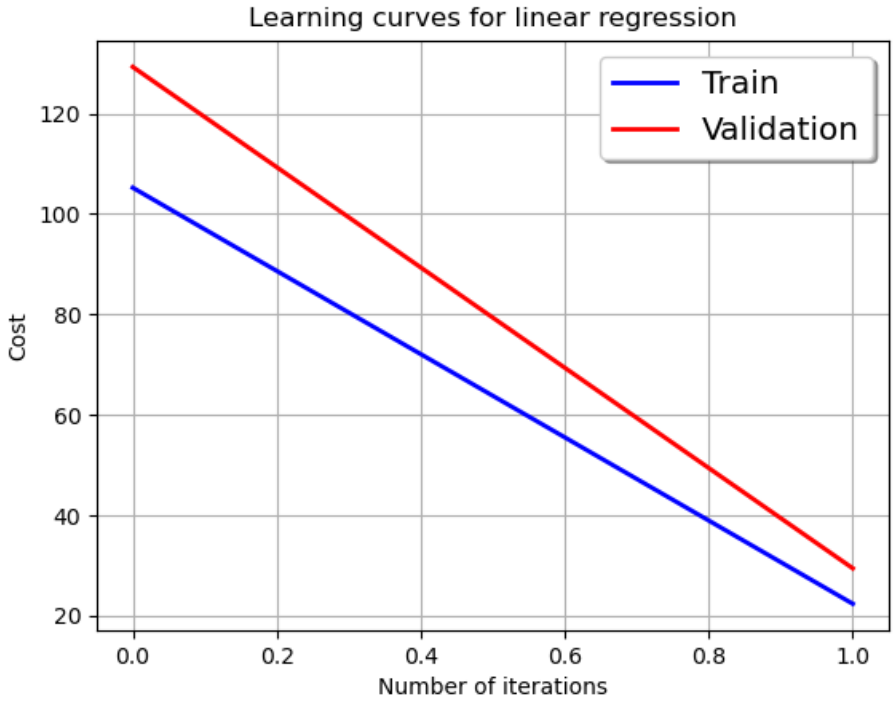
    # Number of training examples
    m, _ = X.shape
    cost_train = 0.
    cost_val = 0.

    _, cost_train, cost_val =trainLinearReg(X, y, Lambda, Xval
       , yval, maxiter=maxiter)

    # display
    plt.figure()
    ax.plot(range(cost_train.size), cost_train, color='b', lw
       =2, label='Train')
    ax.plot(range(cost_val.size), cost_val, color='r', lw=2,
       label='Validation')
    ax.set_title('Learning curves for linear regression')
    ax.grid()
    ax.set_xlabel('Number of iterations')
    ax.set_ylabel('Cost')
    ax.legend(loc='upper right', shadow=True, fontsize='x-
       large', numpoints=1)

    return cost_train, cost_val
```

- Output :



3 Polynomial regression